instance_eval()

Where you learn another way to mix code and bindings at will.

The following program demonstrates BasicObject#instance_eval, which evaluates a block in the context of an object:

blocks/instance_eval.rb

```
class MyClass
  def initialize
    @v = 1
  end
end

obj = MyClass.new

obj.instance_eval do
  self        # => #<MyClass:0x3340dc @v=1>
  @v          # => 1
end
```

The block is evaluated with the receiver as self, so it can access the receiver's private methods and instance variables, such as @v. Even if instance_eval changes self, the block that you pass to instance_eval can still see the bindings from the place where it's defined, like any other block:

```
v = 2
obj.instance_eval { @v = v }
obj.instance_eval { @v }      # => 2
```

The three lines in the previous example are evaluated in the same *Flat Scope (83)*, so they can all access the local variable v—but the blocks are evaluated with the object as self, so they can also access obj's instance variable @v. In all these cases, you can call the block that you pass to instance_eval a *Context Probe*, because it's like a snippet of code that you dip inside an object to do something in there.

Spell: Context Probe

Breaking Encapsulation

At this point, you might be horrified. With a *Context Probe (85)*, you can wreak havoc on encapsulation! No data is private data anymore. Isn't that a Very Bad Thing?

Pragmatically, there are some situations where encapsulation just gets in your way. For one, you might want to take a quick peek inside an object from an irb command line. In a case like this, breaking into the object with instance_eval is often the shortest route.